

PoC Batch 1 Review Gate

A. Executive Verdict

YELLOW

Isoliert, ohne Secrets, ohne Scope-Verletzung, beide PoCs laufen grün (Typecheck + Fixture-Lauf von mir selbst verifiziert). Aber: zwei substanzielle technische Ungenauigkeiten und eine **komplett ungehakte DoD** stehen im Widerspruch zu den „erfolgreich/abgeschlossen“-Aussagen der Reports. Kein Sicherheits- oder Scope-RED — aber die Reports übertreiben die Verlässlichkeit und dürfen nicht ungeprüft als Phase-1-Basis gelten.

B. Merge-Empfehlung

- **Merge nach main: ja** — aber als *Experiment*, nicht als „verifizierte Produktbasis“.
- Bedingungen vor Merge (bzw. spätestens bevor die Reports als Source of Truth für Phase 1 dienen):
 1. Report-Claims entschärfen: „funktioniert zuverlässig“ / „Monumente werden korrekt extrahiert“ ist durch den **y=0-Bug** (siehe G) faktisch widerlegt.
 2. Die **DoD-Checklisten** in beiden README.md sind zu 100 % [] (u. a. „50 Server 24h“, „Wipe an echten Servern getestet“, „Terms-Anfrage abgesendet“). Entweder DoD auf *Fixture-only-Scope* umschreiben oder offen als „nicht erfüllt“ markieren. Aktuell behaupten die Reports Abschluss, während die eigene DoD unerfüllt ist — das ist widersprüchlich.
 3. Klarstellen, dass die Budget-Degradation (siehe F) nur *illustrativ* ist und die genannten Zahlen das Limit **nicht** einhalten.

C. Strukturprüfung

- Alle erwarteten Dateien liegen an den erwarteten Pfaden (Reports unter `experiments/*/reports/`, Docs unter `docs/`). [?](#)
- `git diff --stat main..HEAD`: **nur** `docs/` und `experiments/...` betroffen — 30 Dateien, keine Fremdbereiche. [?](#)
- Markdown-Codeblöcke: alle ````bash`-Fences in beiden Reports und in `MODEL_HANDOFF.md` sauber geschlossen. [?](#)
- Reports sind für spätere Modelle verständlich strukturiert. [?](#)
- Konsistenz-Mängel:
 - `CURRENT_SPRINT.md` sagt „Ausführung von **pnpm** start“, alle anderen Stellen (Reports, Handoff, `package.json`) nutzen **npm**. Kein `pnpm-Lockfile` vorhanden. → Doku-Inkonsistenz.
 - `fixtures/example-server-list-response.json` (`{\"data\": []}`) wird von **keinem** Code referenziert (nur `example-server-response.json` wird geladen). Toter Fixture.

D. Scope-Prüfung

Bestätigt (per Diff verifiziert):

- **Kein** produktiver Code außerhalb /experiments. [?]
- **Keine** Web-App-/Desktop-App-Features. [?]
- **Keine** Rust+-Implementierung. [?]
- **Keine** Discord-Produktintegration. [?]
- **Keine** Billing-Integration. [?]
- **Keine** SEO-Serverseiten. [?]
- rustplus-pairing-poc, map-format-poc, discord-alert-poc unangetastet (nur READMEs aus Phase 0). [?]

E. Secret-Prüfung

- Getrackte .env-Dateien: **nur** battlemetrics-poller-poc/.env.example (BATTLEMETRICS_TOKEN=) und rustmaps-lookup-poc/.env.example (RUSTMAPS_API_KEY=) — **leere Platzhalter**, keine Werte.
- git grep über getrackte Dateien nach API_KEY|TOKEN|SECRET|PASSWORD|PRIVATE_KEY|BEARER mit Wertzuweisung: einziger Treffer this.token = token; (Code, kein Secret).
- .gitignore deckt .env, .env.* (Ausnahme .env.example), secrets/, *.pem, *.key korrekt ab. [?]
- node_modules/ ist **nicht** getrackt (0 Dateien in git). [?]
- **Bewertung: echte Secrets? Nein. Nur Platzhalter. Risiko: niedrig.** Token/Key werden ausschließlich aus process.env (.env.local, gitignored) gelesen — Live-Modus ohne Secret-Commit korrekt umgesetzt.

F. BattleMetrics Review

Geprüft: Parser, Fixture-Modus, Live-Modus, Normalizer, Wipe-Detector, Budget-Simulator.

- **Server-ID/URL-Parser** (server-url-parser.ts): funktioniert für reine IDs und battlemetrics.com/servers/rust/<id>. Sauberer Fehlerpfad. [?] (*Nur der /rust/-Pfad wird erkannt — andere Spiel-Slugs würden fehlschlagen; für Rust-only-Scope ok.*)
- **Fixture-Modus** (kein Token → Fixture): sauber, deterministisch. [?]
- **Live-Modus:** Bearer-Header aus this.token, kein Secret-Commit. Endpoint api.battlemetrics.com/servers/{id} — **nicht verifiziert** (keine Live-Calls; API-Fakten nicht erfunden).

- **Normalizer:** defensive || Fallbacks, rawDetailsKeys als Sicherheitsnetz für variable Felder. Solide. $\Delta \square$ Falsy-||-Muster (players || 0 etc.) hier unkritisch, aber siehe RustMaps für den Realschaden derselben Idiomatik.
- **Wipe-Detector** (wape-detector.ts): **Fixture-Erkenntnis** — schlägt bei rustLastWipe/Seed/Size/Entity-Drop an. Semantik der kind-Klassifizierung ist unscharf: ein reiner rustLastWipe-Wechsel ohne Seed-Change wird als 'full' zurückgegeben, obwohl das real oft ein *Map-only*-Wipe (Forced Wipe mit erhaltenen BPs) ist. Heuristik nie an echten Wipe-Daten validiert (DoD-Punkt ungehakt).
- **Polling-Budget-Simulator** (polling-budget-simulator.ts): **Risiko unterschätzt**. Der Lauf zeigt: nach „Degradation“ bleibt die Rate bei 5.000 Servern bei **617 RPM** und bei 50.000 bei **6.167 RPM** — beides *über* dem 600-RPM-Limit. Die von Report und POLLING_BUDGET_DESIGN.md behauptete Aussage „Degradation schützt die Hot-Server / löst das Problem“ ist mit *diesen* Zahlen **nicht** erfüllt: ein realer Controller müsste Cold härter drosseln/pausieren oder die Hot-Menge deckeln. Aktuell nur illustrative Mathematik, kein tragfähiges Budget-Modell.
- **Distinct-Server-Problem:** nur als *hardcodierte Verteilung* (10/30/60 %) simuliert. Keine echte Dedup-/Distinct-Logik implementiert; die Server-List-Fixture ist leer/ungenutzt. Für einen PoC vertretbar, aber die „zentrale Erkenntnis“ ruht auf Annahmen, nicht auf Daten.
- **Keine Rohdaten-Replikation im Report:** bestätigt (nur Feldnamen, keine kopierten API-Payloads). $\square$
- **Terms-/SEO-Sperre** dokumentiert: $\square$ (Report-Abschnitt „Grenzen / Nicht erlaubt“).
- **Wiederverwendbarkeit Phase 1:** Normalizer + Typen sind wiederverwendbar; Wipe-Heuristik und Budget-Controller sind **Experiment-Stand**, nicht produktreif.

G. RustMaps Review

Geprüft: Input, Fixture/Live, Normalizer, Monumente, Bild-URL, Attribution.

- **Seed/Size-Input + Fixture-Modus + Live-Modus** (X-API-Key, kein Secret-Commit): sauber. Endpoint rustmaps.com/api/v4/maps/{size}/{seed} — **nicht verifiziert** (ich erfinde keine API-Fakten; die Host-/Pfadform ist ungeprüft und sollte im Live-Test bestätigt werden).
- **Bild-URL-Handling:** nur URL-Normalisierung, **keine Downloads, kein Rehosting** — im Code bestätigt (keinerlei fs.write/fetch auf Bilder). $\square$
- **Attribution Notes** (attribution-notes.ts): Pflicht-Backlink + Caching-Sperre klar dokumentiert. $\square$
- $\square$ **Bestätigter Bug (aus meinem Lauf):** Der Normalizer nutzt y: m.y || m.z (map-lookup-normalizer.ts:8). Das Monument „**Outpost**“ mit x:0, y:0 verliert im Output seine Y-Koordinate (0 ist falsy $\rightarrow$ 0 || undefined $\rightarrow$ undefined $\rightarrow$ fällt aus dem JSON). Im tatsächlichen Fixture-Output fehlt bei Outpost das y-Feld. Das widerlegt direkt die Report-Aussage „Monumente ... werden korrekt extrahiert“ bzw. „um Inkonsistenzen in den

Koordinaten abzufangen". Map-Zentrum (0/0) ist ein realistischer Fall. Gleiches ||-Falsy-Risiko bei seed || 0 / size || 0.

- **Monument-Daten:** category→type, grid→gridRef gemappt. Prinzip ok, aber siehe Bug oben.
- **Terms-/Caching-Sperre:** [?](#) dokumentiert.
- **Wiederverwendbarkeit Phase 1:** Struktur ja, aber **erst nach Fix** des Koordinaten-Bugs.

H. Testergebnisse

Von mir **selbst ausgeführt** (kein Vertrauen auf fremde Behauptungen):

PoC	tsc --noEmit	Fixture-start
battlemetrics-poller-poc	EXIT 0 ?	läuft, normalisiert + Wipe-Detect + Budget-Sim ?
rustmaps-lookup-poc	EXIT 0 ?	läuft, normalisiert + Attribution ? (mit y=0-Bug im Output sichtbar)

- npm/tsx/tsc verfügbar, node_modules vorhanden — keine Live-Calls nötig, beide Clients fallen ohne Token/Key automatisch in den Fixture-Modus.
- **Verifikation der Handoff-Behauptung:** MODEL_HANDOFF.md behauptet „Typecheck lief fehlerfrei, npm start lieferte normalisierte Ausgaben“ — das ist **wahr, nicht erfunden**. [?](#)
- pnpm nicht getestet (nicht vorhanden/nicht nötig); die pnpm-Erwähnung in CURRENT_SPRINT.md ist irreführend, aber harmlos.

I. Offene Risiken

1. **Budget-Degradation trägt nicht (F):** die propagierte Lösung hält das Limit nicht ein — echtes Design offen.
2. **y=0-Koordinaten-Bug (G):** Datenverlust bei Zentrums-Monumenten.
3. **API-Endpoints beider Clients nicht verifiziert** (kein Live-Test) — Machbarkeit real ungeprüft.
4. **Wipe-Heuristik nie an echten Wipe-Daten getestet** (DoD-Punkt offen).
5. **Distinct-Server-Dedup nicht implementiert** — nur Annahmen.
6. Falsy-||-Idiom als latente Fehlerquelle in beiden Normalizern.

J. Terms-/Legal-Blocker

Sauber dokumentiert in LEGAL_TERMS_OUTREACH.md, LEGAL_SECURITY_BOUNDARIES.md, API_SOURCES.md, beiden Reports — aber **noch nicht adressiert** (Outreach nicht abgesendet, DoD-Punkte []):

- **BattleMetrics:** kommerzielle Nutzung, SEO-Re-Publikation, zulässiges Polling-Volumen (Enterprise) — **ungeklärt**.
- **RustMaps:** Bild-Caching/Rehosting, exakte Attribution-Vorgaben — **ungeklärt**. Für den *Merge eines isolierten Experiments* sind das **keine** Blocker. Für jede **Produktisierung** sind es **harte Gates**.

K. Muss vor nächstem Schritt repariert werden

(vor Weiterverwendung als Phase-1-Basis — nicht zwingend vor dem reinen Experiment-Merge)

1. **y=0-Bug** im RustMaps-Normalizer (m.y ?? m.z statt ||).
2. **Report-Claims korrigieren:** „zuverlässig/korrekt“ auf den Fixture-Stand herunterstufen; Budget-Degradation als *illustrativ, Limit nicht eingehalten* kennzeichnen.
3. **DoD reconcilieren:** READMEs sind zu 100 % ungehakt und widersprechen den „abgeschlossen“-Reports.

L. Kann später repariert werden

- pnpm→npm-Inkonsistenz in CURRENT_SPRINT.md.
- Ungenutzte/leere example-server-list-response.json entfernen oder befüllen.
- kind-Semantik des Wipe-Detectors (map vs. full) schärfen.
- Falsy-|| in BM-Normalizer defensiv auf ?? umstellen.
- Live-Endpoint-Verifikation beider APIs (im späteren Live-Test).

M. Empfehlung für nächsten Schritt

Reihenfolge: (c) beides parallel, aber strikt getrennt.

- **Terms-Outreach SOFORT starten** — es hat lange externe Latenz und ist der einzige *echte* Blocker für Produktisierung. Nicht auf Batch 2 warten lassen.
- **PoC Batch 2 (Rust+ Pairing, Map-Format) darf parallel starten**, weil er *technisch unabhängig* von den BM/RustMaps-Terms ist — jedoch nur als isolierte Experimente (gleiche Guardrails wie Batch 1), **keine** Produktisierung, bis Terms geklärt sind.
- Nicht empfohlen: Batch 2 *vor* Outreach zu priorisieren (verschenkt die Antwort-Wartezeit); nicht empfohlen: Outreach *vor* Batch 2 seriell (blockiert unnötig, da unabhängig).

N. Finaler Go/No-Go-Beschluss

GO für Merge nach main als isoliertes Experiment (YELLOW) — bewilligt, weil: keine Secrets, keine Scope-Verletzung, node_modules untracked, beide PoCs verifiziert grün.

Aber: PoC Batch 1 **darf noch nicht als „fachlich verifizierte, verlässliche Basis“ gelten**, solange K.1–K.3 offen sind (y=0-Bug, überzogene Report-Claims, ungehakte DoD). Bedingung: Diese drei Punkte

vor Weiterverwendung durch das ausführende Modell beheben — oder den Report-Status explizit auf „Fixture-PoC, unvalidiert“ setzen.

Empfohlene Sequenz nach Merge: Terms-Outreach sofort + Batch 2 parallel (getrennt), Produktisierung erst nach Terms-Klärung.

Keine Dateien geändert, keine Commits, kein Merge, kein Push, keine Live-Calls, keine Secrets verwendet. Review-Gate endet hier.